@aman_views

JUnit & Mockito Notes

Beginner to Advanced

☆ 1. What is JUnit?

JUnit is a testing framework for Java.
It is used to write and run repeatable tests to ensure code quality.

☆ 2. Why JUnit?

- Helps in writing automated tests.
- Detects bugs early.
- Improves code quality.
- Makes refactoring safe.
- Easy to use and integrate.

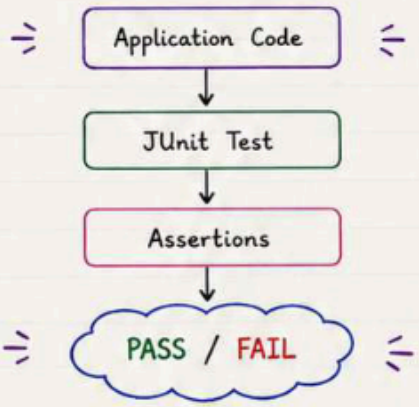
3. Unit Testing vs Integration Testing

Unit Testing	Integration Testing
• Tests individual unit (method/class).	• Tests multiple units together.
• Dependencies are mocked.	• Real dependencies used.
• Fast execution.	• Slower than unit tests.
• Done by Developers.	• Done by Devs/QA.

4. JUnit 4 vs JUnit 5

Feature	JUnit 4	JUnit 5
Package	org.junit	org.junit.jupiter
Annotations	@Test	@Test
Architecture	Monolithic	Modular
Assertions	Limited	Many new assertions
Extensions	Runners	Extensions API
Features	Basic	Advanced (Parameterized, Nested, etc.)

JUnit Test Flow



5. Maven Dependency

JUnit 5 Dependency →

```
<dependency>
    <groupId> org.junit.jupiter </groupId>
    <artifactId> junit-jupiter </artifactId>
    <version> 5.10.0 </version>
    <scope> test </scope>
</dependency>
```

6. Test Class Naming Convention

- Test classes should end with "Test".

Example: CalculatorTest, UserServiceTest

7. Basic @Test Example

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class CalculatorTest {
    @Test
    void addTest() {
        int result = 2 + 3;
        assertEquals(5, result);
    }
}
```

← Simple Test Example



Note: JUnit 5 works on Jupiter Platform.
It supports better features, parallel execution and easy extensibility. ☆

JUnit & Mockito Notes

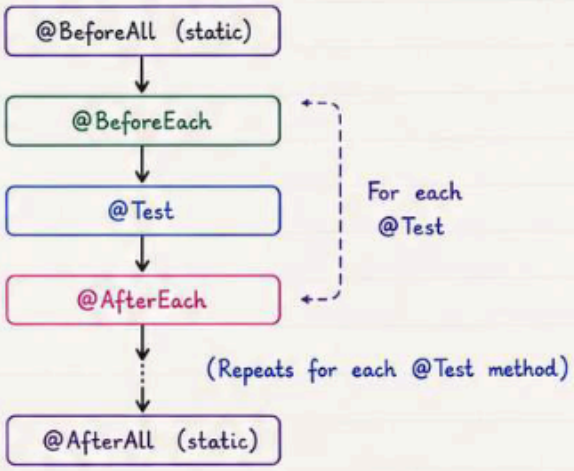
Beginner to Advanced

Page 2 : JUnit 5 Annotations & Test Lifecycle

1. Important Annotations in JUnit 5

Annotation	Purpose
@Test	Marks a method as a test method.
@BeforeEach	Runs before each @Test method.
@AfterEach	Runs after each @Test method.
@BeforeAll	Runs once before all tests in the class. (must be static)
@AfterAll	Runs once after all tests in the class. (must be static)
@DisplayName	Provides a custom name for the test.
@Disabled	Disables the test method.
@Nested	Creates nested test classes.
@Order	Used with @TestMethodOrder to specify test execution order.
@TestMethodOrder	Defines the order for test methods. (Annotation + Orderer)

2. Test Lifecycle in JUnit 5



3. Example : Lifecycle in Action

```
import org.junit.jupiter.api.*;
public class LifecycleExample {
  @BeforeAll
  static void initAll() {
      System.out.println("Before All Tests");
  }
  @BeforeEach
  void setUp() {
      System.out.println("Before Each Test");
  }
  @Test
  void testOne() {
      System.out.println("Test One");
  }
  @Test
  void testTwo() {
      System.out.println("Test Two");
  }
  @AfterEach
  void tearDown() {
      System.out.println("After Each Test");
  }
  @AfterAll
  static void cleanUpAll() {
      System.out.println("After All Tests");
  }
}
```

Output Order

① Before All Tests
↓
② Before Each Test
↓
③ Test One
↓
④ After Each Test
↓
⑤ Before Each Test
↓
⑥ Test Two
↓
⑦ After Each Test
↓
⑧ After All Tests

4. @DisplayName Example

```
@Test
@DisplayName("Addition of two numbers should return 5")
void addTest() {
    int result = 2 + 3;
    Assertions.assertEquals(5, result);
}
```

5. @Disabled Example

```
@Test
@Disabled("This test is disabled temporarily")
void disabledTest() {
    // This test will be skipped
}
```

6. @Nested Example

```
@Nested
@DisplayName("User Service Tests")
class UserServiceTest {
    @Test
    void getUserTest() {
        // test code
    }
}
```



Note: JUnit 5 annotations help you control the test execution flow, improve readability and write clean, maintainable tests.

@aman_views

JUnit & Mockito Notes

Beginner to Advanced

Page 3 : Assertions & Parameterized Tests in JUnit 5

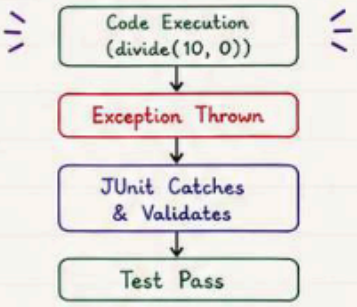
1. Assertions in JUnit 5

Assertion	Purpose	Example
assertEquals(expected, actual)	Checks if two values are equal.	assertEquals(5, result);
assertNotEquals(expected, actual)	Checks if two values are not equal.	assertNotEquals(5, result);
assertTrue(condition)	Checks if condition is true.	assertTrue(age > 18);
assertFalse(condition)	Checks if condition is false.	assertFalse(isEmpty);
assertNull(object)	Checks if object is null.	assertNull(user);
assertNotNull(object)	Checks if object is not null.	assertNotNull(user);
assertThrows(Exception.class, () → {})	Checks if exception is thrown.	assertThrows(ArithmeticException.class, () → divide(10, 0));
assertAll("msg", Executable...)	Executes all assertions and reports all failures.	assertAll("All checks", () → assertEquals(5, sum), () → assertTrue(sum > 0));
assertTimeout(Duration, Executable)	Checks if execution completes in given time.	assertTimeout(Duration.ofSeconds(2), () → service.call());

2. assertThrows() Example

```
@Test
void testDivideByZero() {
    Exception ex = assertThrows(
        ArithmeticException.class,
            () → divide(10, 0)
    );
    assertEquals("/ by zero", ex.getMessage());
}
```

How assertThrows() Works



3. assertAll() Example

```
@Test
void testMultipleConditions() {
    int a = 2 + 3;
    int b = 10;

    assertAll("Multiple Validations",
        () → assertEquals(5, a),
        () → assertTrue(b > a),
        () → assertNotNull(String.valueOf(a))
    );
}
```

Even if one assertion fails, others will still be executed.

4. Parameterized Tests in JUnit 5

Annotation	Purpose
@ParameterizedTest	Marks a method as parameterized test.
@ValueSource	Provides a list of values.
@CsvSource	Provides comma separated values.
@MethodSource	Uses method to provide arguments.
@EnumSource	Uses enum constants.
@NullSource	Provides null values.
@EmptySource	Provides empty values ("", Collections.empty,etc.)

5. Parameterized Test Examples

① @ValueSource

```
@ParameterizedTest
@ValueSource(ints = {1, 2, 3, 4, 5})
void testIsPositive(int num) {
    assertTrue(num > 0);
}
```

② @CsvSource

```
@ParameterizedTest
@CsvSource({"2, 3, 5", "4, 6, 10"})
void testAddition(int a, int b, int expected) {
    assertEquals(expected, a + b);
}
```

③ @MethodSource

```
@ParameterizedTest
@MethodSource("stringProvider")
void testLength(String str, int len) {
    assertEquals(len, str.length());
}
static Stream<Arguments> stringProvider() {
    return Stream.of(
        Arguments.of("Java", 4),
        Arguments.of("JUnit", 5)
    );
}
```

Parameterized tests help run the same test logic with multiple inputs. Saves time & reduces code!

Note: Good tests are independent, repeatable and easy to maintain.
Use meaningful test names and test one behavior at a time.

JUnit & Mockito Notes

Beginner to Advanced

Page 4 : Mockito Basics & Core Annotations

1. What is Mockito?

- Mockito is a popular mocking framework for unit testing in Java.
- It helps us test a class in isolation by mocking its dependencies.
- No need of real database, server or external services.

> **Key Idea**
> Mock the dependencies, test the logic!

2. Why Mockito?

- ☑ Create mock objects easily.
- ☑ Isolate the class under test.
- ☑ Control the behavior of mocks.
- ☑ Verify interactions between objects.
- ☑ Reduce boilerplate code.

3. Mock vs Spy

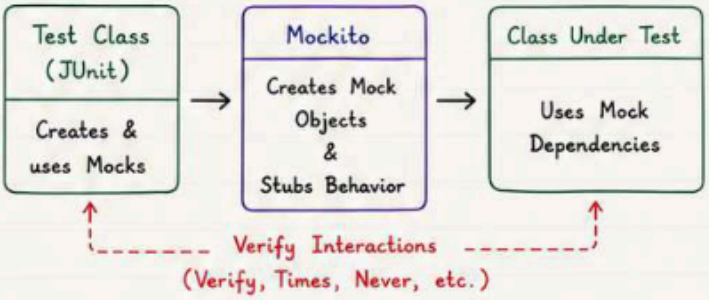
Feature	Mock	Spy
Creates	New object	Wraps real object
Method Calls	Return default value	Call real methods by default
Use when	We don't want real method execution	We want partial real method execution
Example	`when(mock.get()).thenReturn(10);`	`doReturn(10).when(spy).get();`

4. Mockito Annotations

Annotation	Purpose
@Mock	Creates a mock object.
@InjectMocks	Injects mock dependencies into the class under test.
@Spy	Creates a spy object (partial mock).
@Captor	Captures arguments passed to a method.
@BeforeEach	Initializes mocks before each test.

These annotations work with MockitoExtension.

5. Mockito Architecture



6. Basic Mockito Example

```
import org.junit.jupiter.api.*;
import org.mockito.*;
import static org.mockito.Mockito.*;

class UserService {
    private UserRepository userRepository;

    UserService(UserRepository userRepository) {
        this.userRepository = userRepository;
    }

    String getUserName(int id) {
        return userRepository.findById(id);
    }
}

interface UserRepository {
    String findById(int id);
}

class UserServiceTest {
    @Mock
    private UserRepository userRepository;

    @InjectMocks
    private UserService userService;

    @BeforeEach
    void setUp() {
        MockitoAnnotations.openMocks(this);
    }

    @Test
    void testGetUserName() {
        when(userRepository.findById(1)).thenReturn("Aman");

        String name = userService.getUserName(1);

        assertEquals("Aman", name);
        verify(userRepository, times(1)).findById(1);
    }
}
```

> **Flow:**
> Mock Repo
> ↓ stub return
> Service call
> ↓
> Verify result & interaction

> **Note:** Mockito helps in writing clean, fast and reliable unit tests by mocking dependencies and verifying behavior.

JUnit & Mockito Notes

Beginner to Advanced

Page 5 : Stubbing, Verification & Argument Matchers in Mockito

1. Stubbing in Mockito

Stubbing means defining behavior for mock methods.

Stubbing Type	Syntax	Example
when().thenReturn()	when(mock.method()) .thenReturn(value);	when(userRepo.findById(1)) .thenReturn(user);
when().thenThrow()	when(mock.method()) .thenThrow(exception);	when(userRepo.findById(1)) .thenThrow(new RuntimeException());
doReturn().when()	doReturn(value) .when(mock).method();	doReturn(10) .when(calculator).add(2,3);
doThrow().when()	doThrow(exception) .when(mock).method();	doThrow(new IllegalStateException()) .when(calculator).divide(10,0);
thenReturn(f (multiple values)	when(mock.method()) .thenReturn(v1, v2, v3);	when(service.getData()) .thenReturn(d1, d2, d3);

★ Consecutive calls will return values in the order provided.

2. Verification in Mockito

Verification ensures that certain methods were called on the mock objects.

Verification	Syntax	Description
verify(mock)	verify(mock).method();	Method called at least once
times(n)	verify(mock, times(n)) .method();	Method called exactly n times
atLeast(n)	verify(mock, atLeast(n)) .method();	Called at least n times
atMost(n)	verify(mock, atMost(n)) .method();	Called at most n times
never()	verify(mock, never()) .method();	Never called
noMoreInteractions(mock)	verifyNoMoreInteractions(mock)	No more interactions with mock

3. Verification Example

```
@Test
void testSaveUser() {
    userService.saveUser(user);

    verify(userRepo).save(user);              // at least once
    verify(userRepo, times(1)).save(user);    // exactly once
    verify(userRepo, never()).delete(user);   // never called
    verifyNoMoreInteractions(userRepo);       // no extra calls
}
```

4. Argument Matchers

Matchers are used when exact value is not important.

Matcher	Usage	Example
any()	Matches any value.	when(repo.findById(anyInt())) .thenReturn(user);
anyString()	Matches any String.	when(repo.findByName(anyString())) .thenReturn(list);
anyInt(),anyLong()	Matches any int/long.	when(service.getById(anyInt())) .thenReturn(data);
eq(value)	Matches exact value.	when(repo.findByStatus(eq("ACTIVE"))) .thenReturn(list);
isNull()	Matches null.	when(repo.findByName(isNull())) .thenReturn(null);
notNull()	Matches non-null.	when(repo.findByName(notNull())) .thenReturn(list);

5. Using Matchers with verify()

```
verify(userRepo).findById(anyInt());
verify(userRepo, times(2)).updateStatus(eq("ACTIVE"));
verify(userRepo, never()).delete(any());
```

6. Void Methods (doNothing, doThrow)

Method	Syntax	Description
doNothing()	doNothing().when(mock).voidMethod();	Do nothing (default) for void methods
doThrow()	doThrow(exception).when(mock).voidMethod();	Throw exception when void method is called

7. Spy Example

```
List<String> list = new ArrayList<>(List.of("A", "B"));
List<String> spyList = spy(list);
when(spyList.size()).thenReturn(100);     // stub size()
spyList.add("C");                         // real method call
assertEquals(100, spyList.size());        // stubbed
assertEquals(3,   spyList.sizeReal());    // real behavior
```

★ Key Points about Spy

- Spy calls real methods unless stubbed.
- Use when(spy.method()).thenReturn() to stub.
- Useful for partial mocking.

JUnit & Mockito Notes

Beginner to Advanced

Page 6 : Mockito with Spring Boot & Best Practices

1. Mockito with Spring Boot

In Spring Boot tests, we use @MockBean to replace a real bean with a mock in the Application Context.

@Mock vs @MockBean	
@Mock	**@MockBean**
• Used in pure unit tests. • Creates mock object inside the test class only. • No Spring context involved.	• Used in Spring Boot tests. • Replaces a bean in Application Context. • Used with @SpringBootTest, @WebMvcTest, etc.

2. Spring Boot Test Annotations

Annotation	Used For	Loads Context
@SpringBootTest	Full integration test (All beans loaded)	Full Context
@WebMvcTest	Testing Controllers (Web layer only)	Web Layer Only
@DataJpaTest	Testing JPA Repositories (Database layer)	JPA Layer Only
@MockBean	Mocking beans in Spring context	N/A
@AutoConfigureMockMvc	To test REST APIs using MockMvc	Web Layer

3. Example : Controller Test with @MockBean

```
@WebMvcTest(UserController.class)
class UserControllerTest {
    @Autowired
    private MockMvc mockMvc;

    @MockBean
    private UserService userService;

@Test
void getUser_success() throws Exception {
  User user = new User(1, "Aman", "aman@gmail.com");
  when(userService.getUser(1)).thenReturn(user);

  mockMvc.perform(get("/api/users/1"))
         .andExpect(status().isOk())
         .andExpect(jsonPath("$.name").value("Aman"));
}
}
```

Flow:
Controller
↓
Service (Mocked)
↓
Returns Stubbed Data
↓
Verify Response

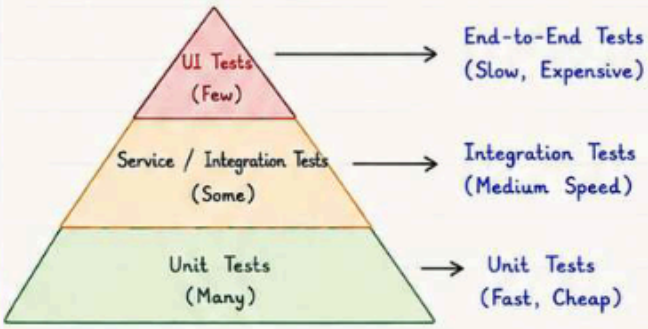
4. Mockito Best Practices

- ☑ Mock only the dependencies, not the class under test.
- ☑ Write tests for one behavior at a time.
- ☑ Use meaningful test method names.
- ☑ Keep tests independent and isolated.
- ☑ Avoid unnecessary stubbing.
- ☑ Prefer constructor injection for easy testing.
- ☑ Use verify() to check interactions.
- ☑ Do not mock value objects or simple getters.

5. Common Mockito Annotations Quick View

@Mock	Creates a mock object.
@InjectMocks	Automatically injects @Mock objects into the target class.
@Spy	Wraps real object and mocks some methods.
@Captor	Captures arguments passed to methods.
@BeforeEach	Runs before each test.
@AfterEach	Runs after each test.

6. Test Pyramid



Final Note

- ☑ JUnit helps you write and organize tests effectively.
- ☑ Mockito helps you isolate the code and test it in a controlled way.
- ☑ Together they make your code reliable, maintainable and bug-free.

Test Well, Code Confidently!